



Université de Sherbrooke

SQL

Relations virtuelles

SQL_05

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Ameni SOUID (ameni.Souid@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

—

Scriptorum/Scriptorum/SQL_05-Vues, version 1.0.0.a, en date du 2026-02-10

— document de travail, ne pas citer —

Sommaire

...

Mise en garde

Le présent document est en cours d'élaboration ; en conséquence, il est incomplet et peut contenir des erreurs.

Historique

diffusion	resp.	description
2026-02-10	CK	Récupération de notes diverses.

Table des matières

Introduction..... 4

1. Présentation..... 4

2. Définition..... 4

3. Modification de données 6

4. Comportement des clés référentielles 7

Conclusion..... 8

Références 9

Définitions 10

Introduction

Le présent document a pour but de présenter le concept de relations vituelles (vues) qu'on retrouve dans SQL.

Dans un premier temps, nous présenterons une grammaire simplifiée, sous-ensemble propre de la grammaire de la norme ISO 9075:2016, avec, au besoin, certaines particularités de PostgreSQL.

1. Présentation

La théorie relationnelle définit deux formes de variables de relation :

- La relation de base est définie par une énumération de tuples.
- La relation virtuelle est définie par une expression relationnelle référant généralement à d'autres relations.

En SQL :

- Une TABLE (table) se veut une représentation d'une relation de base.
- Une VIEW (vue) se veut une représentation d'une relation virtuelle.

2. Définition

```
view_def ::=  
  CREATE [ OR REPLACE ] [M] VIEW name [ ( {column_name [ ... ] } ) ] AS  
  query
```

Une vue est définie grâce à une expression relationnelle, ce qui comprend entre autres l'utilisation des opérateurs relationnels.

Une vue peut être utilisable de la même manière qu'une table.

Exemple — Université

- L'information relative à une note d'évaluation est exprimée sur 100. Les étudiants désirent recevoir l'information en côte.

```
create view ResultatCote (matricule, activité, trimestre, te, note, cote) as
select matricule, activite, trimestre, te, note,
       case
         when note >= 90 then 'A'
         when note between 80 and 89 then 'B'
         when note between 60 and 79 then 'C'
         when note between 40 and 59 then 'D'
         else 'E'
       end as cote
from Resultat;
```

- Plusieurs étudiants désirent recevoir l'information relative à la note totale par activité.
Définir une vue Bulletin (matricule, activité, trimestre, noteT)

Définition

```
create view Bulletin (matricule, activite, trimestre, noteT) as
  select matricule, activite, trimestre, avg(note) as noteT
  from Resultat
  group by matricule, activite, trimestre
  order by noteT;
```

Utilisation

```
select activite, trimestre, noteT
from Bulletin
where matricule = '15113150'
```

Un SGBDR doit garantir cette sémantique, indépendamment de la représentation sous-jacente qu'il adopte. En pratique, il est donc possible de ne stocker aucun tuple (donc aucun attribut) propre à cette vue.

3. Modification de données

Une vue peut être utilisée comme une table. En particulier, il est possible d'y insérer (INSERT) ou d'en retirer (DELETE) des tuples et, dans certains cas, de mettre à jour des tuples (UPDATE).

Lorsqu'on effectue une modification à un tuple dans une vue, il faut en fait modifier le tuple correspondant dans la table.

Pour simplifier l'utilisation des vues, on a recours, le plus souvent, à des automatismes (TRIGGER), que nous étudierons plus tard.

Pour que les modifications soient applicables à partir de la vue, les conditions suivantes doivent être appliquées :

1. L'expression est un SELECT
2. L'expression n'inclut pas DISTINCT
3. Les dénominations de colonnes sont des références simples (pas d'expression)
4. Le FROM ne référence qu'une seule table (ou une seule vue)
5. Le WHERE ne contient pas de sous-requête relative à la table ou à la vue référencée par le FROM
6. L'expression ne contient pas de GROUP BY
7. L'expression ne contient pas de HAVING

Dans le cas d'un INSERT, il faut ajouter une dernière condition, les colonnes absentes de la table ultimement référencée doivent comporter une valeur par défaut.

Plusieurs dialectes nuancent ces conditions de façon non uniforme et non transportable.

Nous verrons comment pallier ceci avec les automatismes (TRIGGER) !

UNIVERSITÉ

Étudiant clé (matricule)

matricule	nom	adresse
15113150	Paul	>Δ ^ς σ ^ς β
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Activité clé (sigle)

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeEvaluation clé(code)

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat clé(matricule, te, activite, trimestre)

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20132	80
15112354	FI	IFT187	20122	78
15113150	TP	IFT159	20132	75
15112354	FI	GMQ103	20122	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

Figure 1. Exemple — Université

4. Comportement des clés référentielles

Il n'est pas possible de définir des contraintes clés sur une vue. Les contraintes des tables sous-jacentes sont considérées.

Toutefois, la matérialisation d'une vue permettra de contourner ce défaut.

Conclusion

Références

[Elmasri2016]

Ramez ELMASRI et Shamkant B. NAVATHE;
Fundamentals of database systems;
7th Edition, Pearson, Hoboken (NJ, US), 2016;
ISBN 978-0-13-397077-7.

[Date2015a]

Chris J. DATE;
SQL and relational theory: how to write accurate SQL code;
O'Reilly Media, 2015;
ISBN 978-1-4919-4117-1.

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-10-07 18:26:58 UTC



Université de Sherbrooke